Unit-10 Configuration Management

Introduction to Version Management, Change Management and Release Management

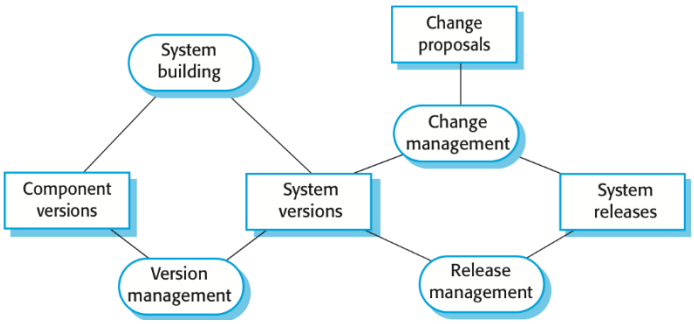
1. Version management
2. System building
3. Change management
4. Release management

Configuration Management (CM)

Software systems are constantly changing during development and use. **Configuration management** (CM) is concerned with the policies, processes and tools for managing changing software systems. You need CM because it is easy to lose track of what changes and component versions have been incorporated into each system version. CM is essential for team projects to control changes made by different developers.

Configuration management activities:

- **Version management:** Keeping track of the multiple versions of system components and ensuring that changes made to components by different developers do not interfere with each other. GitHub is a highly used software that is typically used for version control. It is helpful when more than just one person is working on a project.
- **System building:** The process of assembling program components, data and libraries, then compiling these to create an executable system.
- **Change management:** Keeping track of requests for changes to the software from customers and developers, working out the costs and impact of changes, and deciding the changes should be implemented.
- **Release management:** Preparing software for external release and keeping track of the system versions that have been released for customer use.

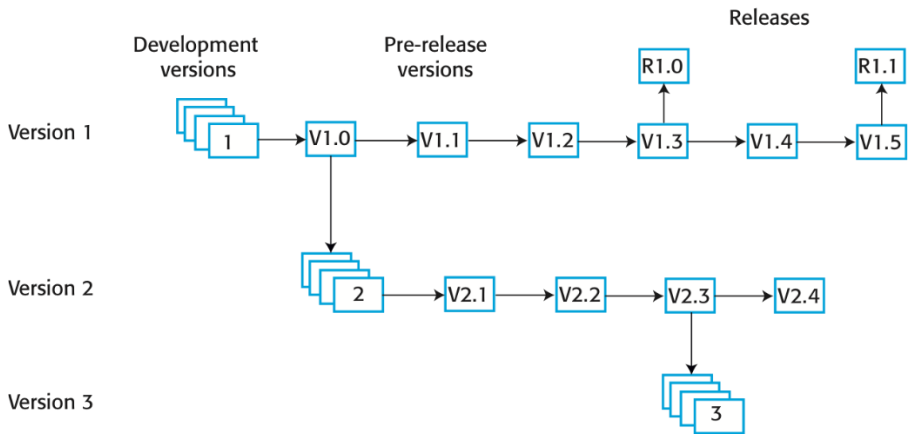


Agile development, where components and systems are changed several times per day, is **impossible without using CM tools**. The definitive versions of components are held in a shared project repository and developers copy these into their own workspace. They make changes to the code then use system building tools to create a new system on their own computer for testing. Once they are happy with the changes made, they return the modified components to the project repository.

- A **development phase** is where the development team is responsible for managing the software configuration and new functionality is being added to the software.
- A **system testing phase** is where a version of the system is released internally for testing. No new system functionality is added in this phase. Changes made are bug fixes, performance improvements and security vulnerability repairs.

- **A release phase** where the software is released to customers for use. New versions of the released system are developed to repair bugs and vulnerabilities and to include new features

For large systems, there is never just one 'working' version of a system. There are always several versions of the system at different stages of development. There may be several teams involved in the development of different system versions.



Version management

Version management (VM) is the process of **keeping track of different versions** of software components or configuration items and the systems in which these components are used. It also involves ensuring that changes made by different developers to these versions do not interfere with each other. Therefore version management can be thought of as the process of managing codelines and baselines.

A **codeline** is a sequence of versions of source code with later versions in the sequence derived from earlier versions. Codelines normally apply to components of systems so that there are different versions of each component.

A **baseline** is a definition of a specific system. The **baseline** therefore specifies the component versions that are included in the system plus a specification of the libraries used, configuration files, etc.

Baselines may be specified using a configuration language, which allows you to define what components are included in a version of a particular system. Baselines are important because you often have to recreate a specific version of a complete system. For example, a product line may be instantiated so that there are individual system versions for different customers. You may have to recreate the version delivered to a specific customer if, for example, that customer reports bugs in their system that have to be repaired.

Version control (VC) systems identify, store and control access to the different versions of components. There are two types of modern version control system.

- **Centralized** systems, where there is a single master repository that maintains all versions of the software components that are being developed. Subversion is a widely used example of a centralized VC system.
- **Distributed** systems, where multiple versions of the component repository exist at the same time. Git is a widely-used example of a distributed VC system. Git is a version control system. Git helps you keep track of code changes.

Key features of version control systems:

- Version and release identification
- Change history recording
- Support for independent development
- Project support
- Storage management

To support independent development without interference, version control systems use the concept of a **project repository** and a **private workspace**. The project repository maintains the 'master' version of all components. It is used to create baselines for system building. When modifying components, developers copy (check-out) these from the repository into their workspace and work on these copies. When they have finished their changes, the changed components are returned (checked-in) to the repository. Developers check out components or directories of components from the project repository into their private workspace and work on these copies in their private workspace. When their changes are complete, they check-in the components back to the repository. If several people are working on a component at the same time, each check it out from the repository. If a component has been checked out, the VC system warns other users wanting to check out that component that it has been checked out by someone else.

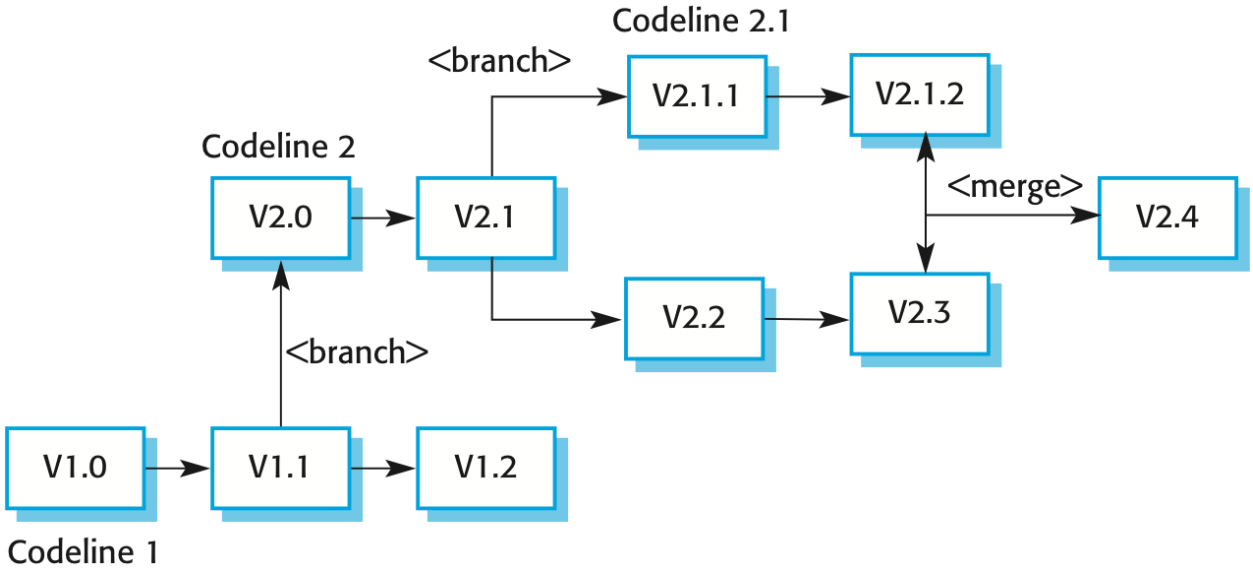
A **'master' repository** is created on a server that maintains the code produced by the development team. Instead of checking out the files that they need, a developer creates a clone of the project repository that is downloaded and installed on their computer. Developers work on the files required and maintain the new versions on their private repository on their own computer. When changes are done, they **'commit'** these changes and update their private server repository. They may then **'push'** these changes to the project repository.

Benefits of distributed version control:

- It provides a **backup mechanism** for the repository. If the repository is corrupted, work can continue and the project repository can be restored from local copies.
- It allows for **off-line working** so that developers can commit changes if they do not have a network connection.
- Project support is the **default way of working**. Developers can compile and test the entire system on their local machines and test the changes that they have made.

Distributed version control is essential for **open source development**. Several people may be working simultaneously on the same system without any central coordination. As well as a private repository on their own computer, developers also maintain a public server repository to which they push new versions of components that they have changed. It is then up to the open-source system 'manager' to decide when to pull these changes into the definitive system.

Rather than a linear sequence of versions that reflect changes to the component over time, there may be several independent sequences resulting from **branching**. This is normal in system development, where different developers work independently on different versions of the source code and so change it in different ways. At some stage, it may be necessary to **merge** codeline branches to create a new version of a component that includes all changes that have been made. If the changes made involve different parts of the code, the component versions may be merged automatically by combining the deltas that apply to the code.



When version control systems were first developed, **storage management** was one of their most important functions. Disk space was expensive and it was important to minimize the disk space used by the different copies of components. Instead of keeping a complete copy of each version, the system stores a list of differences (deltas) between one version and another. By applying these to a master version (usually the most recent version), a target version can be recreated.

As disk storage is now relatively cheap, **Git** uses an alternative, faster approach. Git does not use deltas but applies a standard compression algorithm to stored files and their associated meta-information. It does not store duplicate copies of files. Retrieving a file simply involves decompressing it, with no need to apply a chain of operations. Git also uses the notion of packfiles where several smaller files are combined into an indexed single file.

System building

System building is the process of **creating a complete, executable system** by compiling and linking the system components, external libraries, configuration files, etc. System building tools and version management tools must communicate as the build process involves checking out component versions from the repository managed by the version management system. The configuration description used to identify a baseline is also used by the system building tool.

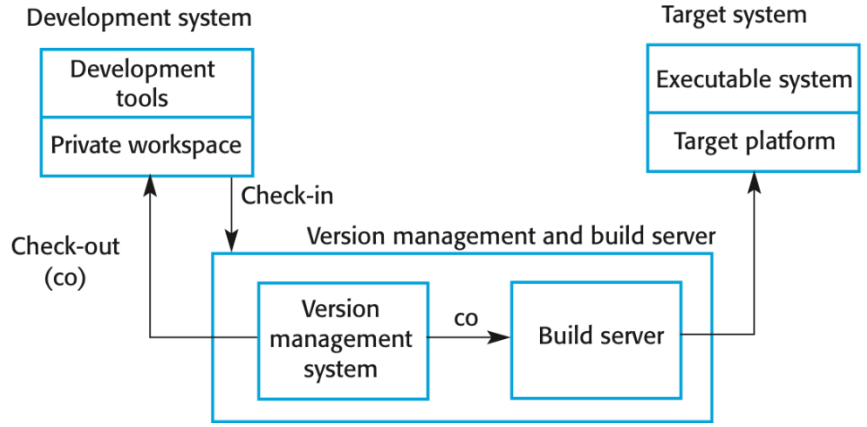
Build platforms may include:

- The **development system**, which includes development tools such as compilers, source code editors, etc. Developers check out code from the version management system into a private workspace before making changes to the system.
- The **build server**, which is used to build definitive, executable versions of the system. Developers check-in code to the version management system before it is built. The system build may rely on external libraries that are not included in the version management system.
- The **target environment**, which is the platform on which the system executes.

Build system functionality includes:

- Build script generation
- Version management system integration
- Minimal re-compilation
- Executable system creation
- Test automation
- Reporting
- Documentation generation

A system platform is the development system, which includes development tools such as compilers, source code editors, etc. The build server, which is used to build definitive, executable versions of the system. This server maintains the definitive versions of a system. The target environment, which is the platform on which the system executes. For real-time and embedded systems, the target environment is often smaller and simpler than the development environment (e.g. a cell phone).



Agile building process (continuous integration):

- Check out the mainline system from the version management system into the developer's private workspace.
- Build the system and run automated tests to ensure that the built system passes all tests. If not, the build is broken and you should inform whoever checked in the last baseline system. They are responsible for repairing the problem.
- Make the changes to the system components.
- Build the system in the private workspace and rerun system tests. If the tests fail, continue editing.
- Once the system has passed its tests, check it into the build system but do not commit it as a new system baseline.
- Build the system on the build server and run the tests. You need to do this in case others have modified components since you checked out the system. If this is the case, check out the components that have failed and edit these so that tests pass on your private workspace.
- If the system passes its tests on the build system, then commit the changes you have made as a new baseline in the system mainline.

Advantages of continuous integration:

- It allows problems caused by the interactions between different developers to be discovered and repaired as soon as possible.
- The most recent system in the mainline is the definitive working system.

Disadvantages of continuous integration:

- If the system is very large, it may take a long time to build and test, especially if integration with other application systems is involved.
- If the development platform is different from the target platform, it may not be possible to run system tests in the developer's private workspace.

Change management

Organizational needs and requirements change during the lifetime of a system, bugs have to be repaired and systems have to adapt to changes in their environment. Change management is intended to **ensure that system evolution is a managed process** and that **priority is given to the most urgent and cost-effective changes**. The change management process is concerned with analyzing the costs and benefits of proposed changes, approving those changes that are worthwhile and tracking which components in the system have been changed.

Factors in change analysis:

- The consequences of not making the change
- The benefits of the change
- The number of users affected by the change
- The costs of making the change
- The product release cycle

In some **agile** methods, **customers are directly involved** in change management. The propose a change to the requirements and work with the team to assess its impact and decide whether the change should take priority over the features planned for the next increment of the system. Changes to improve the software improvement are decided by the programmers working on the system. Refactoring, where the software is continually improved, is not seen as an overhead but as a necessary part of the development process.

Release management

A system release is a version of a software system that is distributed to customers. For mass market software, it is usually possible to identify two types of release: major releases which deliver significant new functionality, and minor releases, which repair bugs and fix customer problems that have been reported. For custom software or software product lines, releases of the system may have to be produced for each customer and individual customers may be running several different releases of the system at the same time. As well as the the executable code of the system, a release may also include:

- Configuration files defining how the release should be configured for particular installations;
- Data files, such as files of error messages, that are needed for successful system operation;
- An installation program that is used to help install the system on target hardware;
- Electronic and paper documentation describing the system;
- Packaging and associated publicity that have been designed for that release.

Factors influencing system release planning:

- **Competition** For mass-market software, a new system release may be necessary because a competing product has introduced new features and market share may be lost if these are not provided to existing customers.
- **Marketing requirements** The marketing department of an organization may have made a commitment for releases to be available at a particular date.
- **Platform changes** You may have to create a new release of a software application when a new version of the operating system platform is released.
- **Technical quality of the system** If serious system faults are reported which affect the way in which many customers use the system, it may be necessary to issue a fault repair release. Minor system faults may be repaired by issuing patches (usually distributed over the Internet) that can be applied to the current release of the system.

Release creation process:

- The executable code of the programs and all associated data files must be identified in the version control system.
- Configuration descriptions may have to be written for different hardware and operating systems.
- Update instructions may have to be written for customers who need to configure their own systems.
- Scripts for the installation program may have to be written.
- Web pages have to be created describing the release, with links to system documentation.
- When all information is available, an executable master image of the software must be prepared and handed over for distribution to customers or sales outlets.

Release tracking In the event of a problem, it may be necessary to reproduce exactly the software that has been delivered to a particular customer. When a system release is produced, it must be documented to ensure that it can be re-created exactly in the future. This is particularly important for customized, long-lifetime embedded systems, such as those that control complex machines. Customers may use a single release of these systems for many years and may require specific changes to a particular software system long after its original release date.

Release reproduction To document a release, you have to record the specific versions of the source code components that were used to create the executable code. You must keep copies of the source code files, corresponding executables and all data and configuration files. You should also record the versions of the operating system, libraries, compilers and other tools used to build the software.

Release planning As well as the technical work involved in creating a release distribution, advertising and publicity material have to be prepared and marketing strategies put in place to convince customers to buy the new release of the system. If releases are too frequent or require hardware upgrades, customers may not move to the new release, especially if they have to pay for it. If system releases are too infrequent, market share may be lost as customers move to alternative systems.

Delivering software as a service (SaaS) reduces the problems of release management. It simplifies both release management and system installation for customers. The software developer is responsible for replacing the existing release of a system with a new release and this is made available to all customers at the same time.

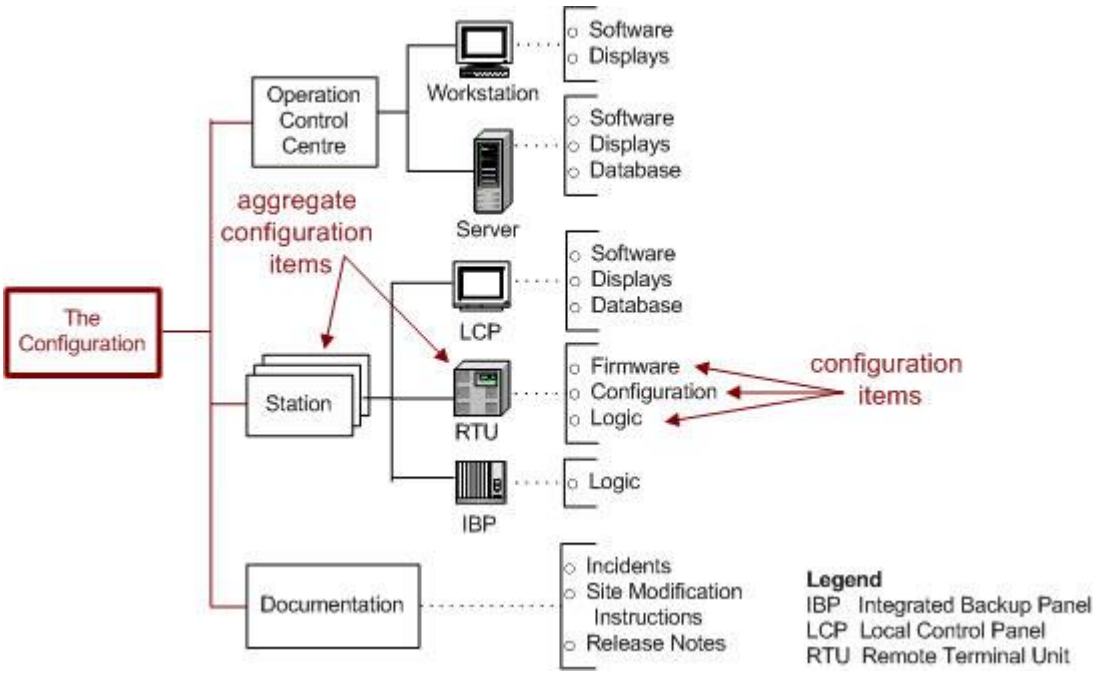
Configuration Items

A component of a system that is treated as a self-contained unit for the purposes of identification and change control. All configuration items (CIs) are uniquely identified by CI registration codes and version

numbers. A CI may be a primitive system building block (e.g. code module) or an aggregate of other CIs (e.g. a sub-system is an aggregate of software units).

A configuration item is a component of a system that can be identified as a self-contained unit for purposes of change control and identification. In other words, version numbers and configuration item registration codes help in uniquely identifying configuration items. Configuration items play an important role in configuration management systems. The versions and changes of configuration items form a major part of any configuration audits.

Configuration items have specific attributes as well as relationships that are often unique for configuration items underneath them in the particular system.

Configuration items help in identifying the components of a system. In configuration management it helps in tracking the granular changes and helps in system maintenance as well for any possible error detection.

Examples of Configuration Items: